

PHASE 12 — QUANTUM RISK ENGINE, MOSCA

MODEL & ENTERPRISE RISK SCORING

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 12 of 22

Document Type: Technical Research & Engineering Handbook

Status: Quantum Risk & Migration Prioritization Architecture

Table of Contents

1. Purpose of This Phase
2. Why Discovery Alone Is Not Enough
3. The Actual Question ECDAT Must Answer
4. Classical Risk vs Quantum Risk
5. Quantum Threat Model
6. Shor's Algorithm
7. Grover's Algorithm
8. Public-Key Cryptography Risk
9. Symmetric Cryptography Risk
10. Hash Function Risk
11. Harvest Now, Decrypt Later
12. Data Lifetime
13. Cryptographic Lifetime
14. Migration Time
15. Quantum Threat Horizon
16. Mosca's Inequality
17. Applying Mosca to ECDAT
18. Mosca Inputs
19. Data Lifetime Estimation
20. Migration Time Estimation
21. Quantum Timeline Uncertainty

22. Risk Categories
23. Algorithm Risk
24. Key-Size Risk
25. Protocol Risk
26. Certificate Risk
27. Library Risk
28. Configuration Risk
29. Custom Crypto Risk
30. Business Criticality
31. Data Sensitivity
32. Exposure
33. Dependency Centrality
34. Migration Complexity
35. Risk Scoring Architecture
36. Risk Dimensions
37. Risk Score Formula
38. Why a Single Score Is Dangerous
39. Multi-Axis Risk
40. Quantum Risk Score
41. Business Impact Score
42. Migration Difficulty Score
43. Exposure Score
44. Confidence Score
45. Priority Score
46. Risk Classification
47. Critical
48. High
49. Medium
50. Low
51. Unknown
52. Risk Aggregation
53. Asset-Level Risk
54. Application-Level Risk

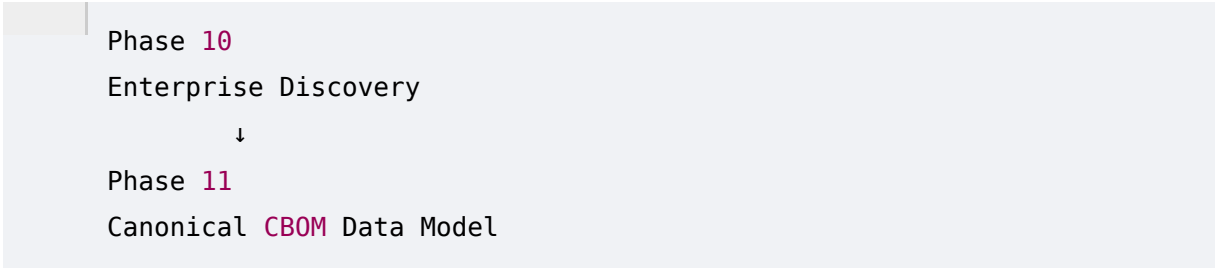
- 55. Business-Level Risk
- 56. Enterprise-Level Risk
- 57. Risk Propagation
- 58. Dependency Risk
- 59. Shared Infrastructure Risk
- 60. PKI Risk
- 61. HSM Risk
- 62. Cloud KMS Risk
- 63. Data-Centric Risk
- 64. Scenario Analysis
- 65. What-If Analysis
- 66. Migration Prioritization
- 67. Priority Matrix
- 68. P1/P2/P3/P4
- 69. Risk Register
- 70. Risk Explanation
- 71. Evidence
- 72. Confidence
- 73. Risk Changes
- 74. Historical Risk
- 75. Risk Dashboard
- 76. Executive View
- 77. Technical View
- 78. Developer View
- 79. Risk API
- 80. Risk Engine Architecture
- 81. Complete Risk Pipeline
- 82. Example 1
- 83. Example 2
- 84. Example 3
- 85. Example 4
- 86. Example 5
- 87. Requirements Derived From Phase 12

88. Phase 12 Summary

89. Phase 13 Preview

1. Purpose of This Phase

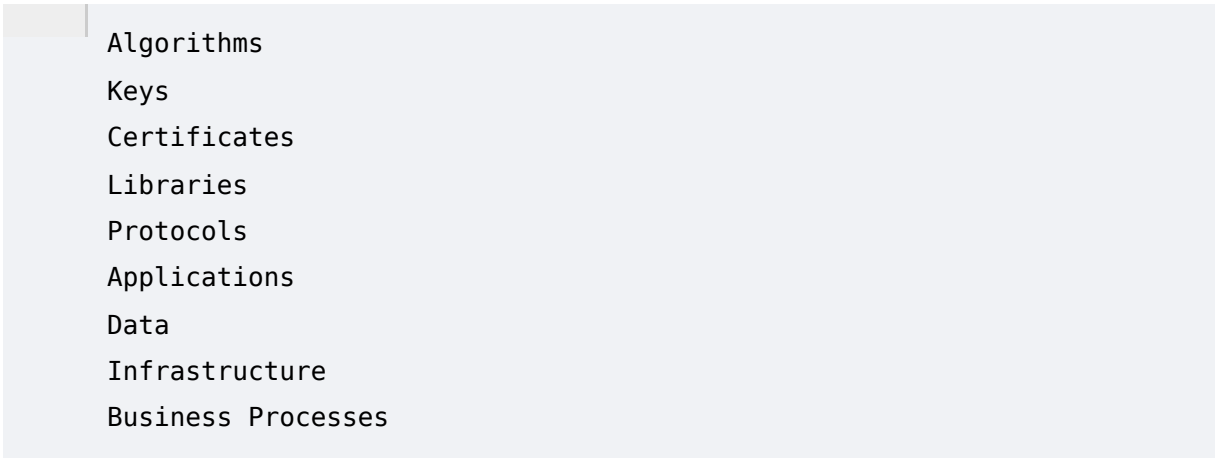
The previous phases established:



```
graph TD; A[Phase 10  
Enterprise Discovery] --> B[Phase 11  
Canonical CBOM Data Model];
```

Phase 10
Enterprise Discovery
↓
Phase 11
Canonical CBOM Data Model

Now ECDAT has knowledge about:



- Algorithms
- Keys
- Certificates
- Libraries
- Protocols
- Applications
- Data
- Infrastructure
- Business Processes

But the NTRO problem statement does not ask merely:

"Find cryptography."

It explicitly asks ECDAT to:

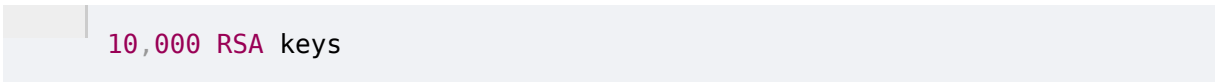
perform a comprehensive quantum risk assessment and identify systems prone to potential quantum attacks, and highlight risks to sensitive data.

Therefore the next layer is:

Risk Intelligence.

2. Why Discovery Alone Is Not Enough

Imagine ECDAT discovers:



10,000 RSA keys

That information alone does not tell the organization what to do.

Consider:

RSA-2048

used for:

Temporary Development Certificate

versus:

RSA-2048

used for:

Long-lived government records

These should not receive identical priority.

Risk requires context.

3. The Actual Question ECDAT Must Answer

ECDAT should ultimately answer:

Which cryptographic assets represent meaningful quantum risk, what systems and data are affected, how urgent is the risk, and what should be migrated first?

This requires combining:

Cryptography

+

Quantum Threat

+

Data Lifetime

+

Migration Time

+

Business Criticality

+

Exposure

+

Dependency



Confidence

4. Classical Risk vs Quantum Risk

ECDAT should distinguish two dimensions.

Classical Risk

Examples:

```
Weak key
Broken algorithm
Expired certificate
Bad configuration
Known vulnerable library
```

Quantum Risk

Examples:

```
RSA
ECC
ECDSA
ECDH
DH
```

potentially threatened by sufficiently capable quantum computing.

The same asset can have:

```
Classical Risk:
Low

Quantum Risk:
High
```

This distinction is essential.

5. Quantum Threat Model

ECDAT should model the major algorithmic threats.

The two most important quantum algorithms are:

```
Shor's Algorithm
Grover's Algorithm
```

They affect cryptographic systems differently.

6. Shor's Algorithm

Shor's algorithm provides a theoretical polynomial-time approach to problems underlying major public-key cryptosystems.

It threatens systems based on:

```
Integer Factorization
Discrete Logarithms
Elliptic-Curve Discrete Logarithms
```

Therefore ECDAT should flag:

```
RSA
DH
ECDH
DSA
ECDSA
EdDSA
```

as quantum-vulnerable categories.

This does **not** mean they are broken by today's classical computers.

The risk is tied to the emergence of sufficiently capable cryptographically relevant quantum computers.

7. Grover's Algorithm

Grover's algorithm provides a quadratic speedup for generic search.

Its implication is different.

For symmetric cryptography, the effective brute-force security is often described approximately as being reduced from:

```
n bits
```

toward:

n/2 bits

under an idealized Grover model.

Therefore:

AES-128

has a substantially different quantum posture than:

AES-256

ECDAT should therefore not label all symmetric cryptography simply as:

Quantum Vulnerable

The assessment needs to be more nuanced.

8. Public-Key Cryptography Risk

For ECDAT's risk engine:

RSA
ECC
DH
DSA
ECDSA
ECDH

should generally be treated as:

Quantum-Vulnerable

for long-term planning purposes.

PQC alternatives can then be recommended.

9. Symmetric Cryptography Risk

Symmetric cryptography generally faces a different quantum threat model.

For example:

AES-128

and:

AES-256

should not receive the same quantum classification.

The risk engine should account for:

Key Size
Security Level
Data Lifetime
Threat Horizon

10. Hash Function Risk

Hash functions are also affected differently.

ECDAT should distinguish:

Preimage Resistance
Collision Resistance
Quantum Search Advantage

and avoid simplistic statements such as:

"SHA-256 is broken by quantum computers."

A more accurate interpretation is:

Quantum algorithms change the security margin of certain cryptographic properties, but do not make every hash function simply "broken."

11. Harvest Now, Decrypt Later

One of the most important concepts in post-quantum migration is:

Harvest Now, Decrypt Later.

An attacker may:

TODAY
↓
Capture encrypted data
↓
Store it
↓

```
Wait for future quantum capability
```

↓

```
Attempt decryption later
```

This means future quantum capability can create a present-day risk for data that needs long-term confidentiality.

12. Data Lifetime

Suppose:

```
Data created:
```

```
2026
```

and the organization needs it confidential until:

```
2041
```

Then:

```
Data Lifetime = 15 years
```

This is one of the most important variables in Mosca-style risk assessment.

13. Cryptographic Lifetime

A cryptographic asset may remain deployed for:

```
5 years
```

```
10 years
```

```
15 years
```

even if the algorithm is expected to become unsuitable earlier.

ECDAT should track:

```
Deployment Date
```

```
Expected Retirement
```

```
Renewal Cycle
```

```
Migration Window
```

14. Migration Time

Migration is rarely instantaneous.

A major enterprise system may require:

```
Architecture
↓
Library Upgrade
↓
Testing
↓
Interoperability
↓
Certification
↓
Deployment
↓
Monitoring
```

Therefore migration may take years.

ECDAT should estimate or allow organizations to provide:

```
Estimated Migration Time
```

15. Quantum Threat Horizon

The exact arrival date of a cryptographically relevant quantum computer is uncertain.

ECDAT should therefore avoid:

```
Quantum Computer Arrives:
2037
```

as though it were a known fact.

Instead it should represent:

```
Threat Horizon:
Scenario-based
```

For example:

```
Optimistic
Baseline
Accelerated
```

16. Mosca's Inequality

A commonly cited formulation associated with Michele Mosca's work is:

$$\lceil X + Y \rceil > Z$$

where:

- **X** = time data needs to remain secure
- **Y** = time required to migrate
- **Z** = time until cryptographically relevant quantum capability is expected

If:

$$\lceil X + Y \rceil > Z$$

then migration should be treated as urgent.

17. Applying Mosca to ECDAT

Example:

Data Lifetime:

12 years

Migration Time:

5 years

Quantum Threat Horizon:

15 years

Then:

$$12 + 5 = 17$$

and:

$$17 > 15$$

Therefore:

Mosca Assessment:

ACTION REQUIRED

This does not mean a quantum computer will definitely arrive in 15 years.

It means the organization's protection and migration timelines overlap in a way that creates strategic urgency.

18. Mosca Inputs

ECDAT should store:

```
data_lifetime
migration_time
threat_horizon
scenario
assessment_date
confidence
```

Example:

```
{
  "data_lifetime_years": 12,
  "migration_time_years": 5,
  "threat_horizon_years": 15,
  "scenario": "baseline",
  "result": "action_required"
}
```

19. Data Lifetime Estimation

Data lifetime can come from:

```
Retention Policy
Regulation
Business Owner
Data Classification
Contract
System Configuration
```

ECDAT should prefer explicit organizational metadata.

AI should not simply guess:

```
"This data probably lasts 20 years."
```

Instead:

Data Lifetime:
Unknown

until evidence exists.

20. Migration Time Estimation

Migration time can come from:

Historical Projects
Engineering Estimates
Application Complexity
Dependency Count
Vendor Support
Testing Requirements
Certification Requirements

Possible categories:

< 1 year
1–3 years
3–5 years
> 5 years
Unknown

21. Quantum Timeline Uncertainty

ECDAT should model scenarios.

Example:

Scenario	Threat Horizon
Conservative	25 years
Baseline	15 years
Accelerated	10 years

These are **planning scenarios**, not predictions.

The user should be able to change them.

22. Risk Categories

ECDAT can classify risk into:

- Quantum Risk
- Classical Cryptographic Risk
- Data Risk
- Business Risk
- Operational Risk
- Migration Risk
- Dependency Risk
- Compliance Risk

23. Algorithm Risk

Algorithm-level assessment:

- ECDSA**
Quantum Status:
Vulnerable
- AES-256**
Quantum Status:
Reduced Security Margin
- SHA-256**
Quantum Status:
Context Dependent

This becomes the first input to application-level risk.

24. Key-Size Risk

The same algorithm can have different security profiles.

Example:

- RSA-1024**

versus:

RSA-4096

The classical and migration posture differs.

ECDAT should therefore assess:

Algorithm

+

Parameter

+

Key Size

rather than algorithm name alone.

25. Protocol Risk

A protocol can introduce additional risk.

For example:

TLS configuration

may use:

Legacy Version

Weak Cipher

Quantum-vulnerable Key Exchange

Therefore risk should be calculated at protocol configuration level.

26. Certificate Risk

Certificate risk can depend on:

Public Key Algorithm

Signature Algorithm

Key Size

Validity Period

Issuer

Purpose

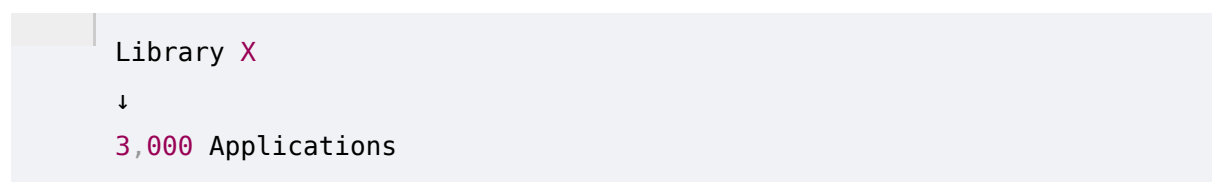
Application Criticality

A long-lived certificate using quantum-vulnerable public-key cryptography can deserve higher migration priority.

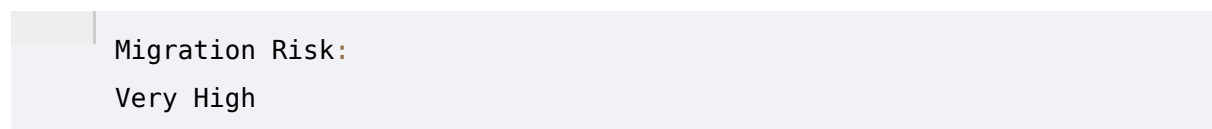
27. Library Risk

A cryptographic library can create systemic risk.

Example:



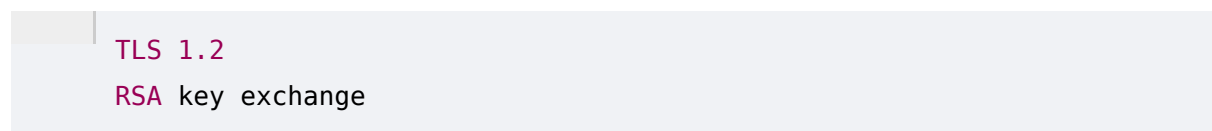
If migration requires upgrading Library X:



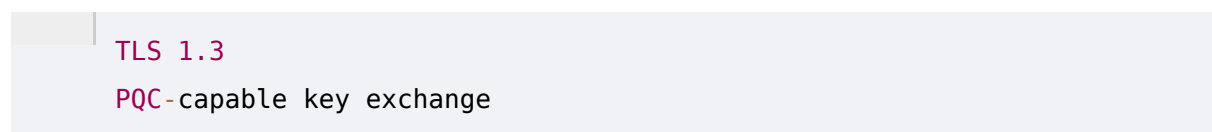
The graph's centrality metrics should therefore influence risk.

28. Configuration Risk

Example:



might create a different posture from:

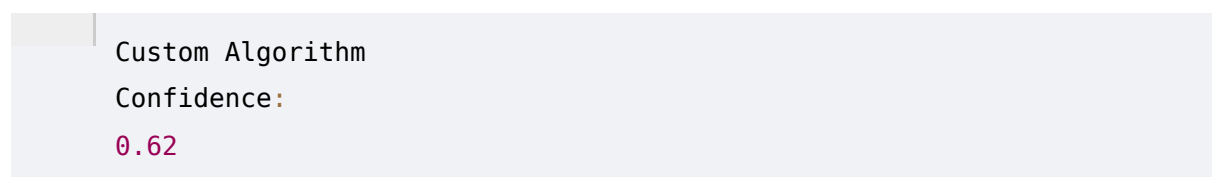


The configuration itself must be assessed.

29. Custom Crypto Risk

Custom cryptography should often receive elevated review priority.

Example:



ECDAT cannot reliably classify the algorithm.

Therefore:

```
Assessment:
Unknown / Review Required
```

may be more appropriate than assigning a false quantum classification.

30. Business Criticality

Risk should consider:

```
Low
Medium
High
Critical
```

Example:

```
Development Application:
Low

Payment Infrastructure:
Critical
```

The same cryptographic weakness can have different organizational consequences.

31. Data Sensitivity

Data classification matters.

For example:

```
Public Data
```

has a very different long-term confidentiality concern than:

```
Strategic Government Data
```

ECDAT should incorporate organizational data classifications.

32. Exposure

Exposure measures how accessible the affected system is.

Potential dimensions:

- Internet-facing
- Partner-facing
- Internal
- Isolated
- Offline

A public-facing service may deserve faster attention than an isolated archive, depending on the threat scenario.

33. Dependency Centrality

From Phase 9:

- Library X
 - ↓
 - 2,000 applications

This creates migration concentration.

The risk engine should incorporate:

- Dependency Centrality

because a vulnerability or migration blocker affecting one central component can have enterprise-wide consequences.

34. Migration Complexity

A migration may be difficult because of:

- Legacy systems
- Vendor dependencies
- Hardware limitations
- Protocol incompatibility
- Performance
- Certification
- Interoperability
- Client dependencies

Therefore:

Risk $\neq$ Only Algorithm Weakness

There is also:

Migration Risk

35. Risk Scoring Architecture

The risk engine should not be one giant formula.

Instead:

```
Evidence
↓
Technical Assessment
↓
Quantum Assessment
↓
Business Context
↓
Migration Assessment
↓
Risk Dimensions
↓
Priority
```

36. Risk Dimensions

ECDAT can calculate multiple dimensions:

```
Quantum Exposure
Classical Security
Data Sensitivity
Business Criticality
Exposure
Dependency Centrality
Migration Complexity
Confidence
```

37. Risk Score Formula

A possible conceptual model:

$$R = f(Q, D, B, E, C, M)$$

where:

- **Q** = quantum risk
- **D** = data sensitivity/lifetime
- **B** = business criticality
- **E** = exposure
- **C** = dependency centrality
- **M** = migration complexity

The exact weighting should be configurable and validated.

38. Why a Single Score Is Dangerous

Suppose:

```
Asset A:  
Risk = 90
```

```
Asset B:  
Risk = 90
```

They may have completely different reasons.

Therefore ECDAT should display:

```
Overall Priority:  
90
```

```
Quantum Risk:  
95
```

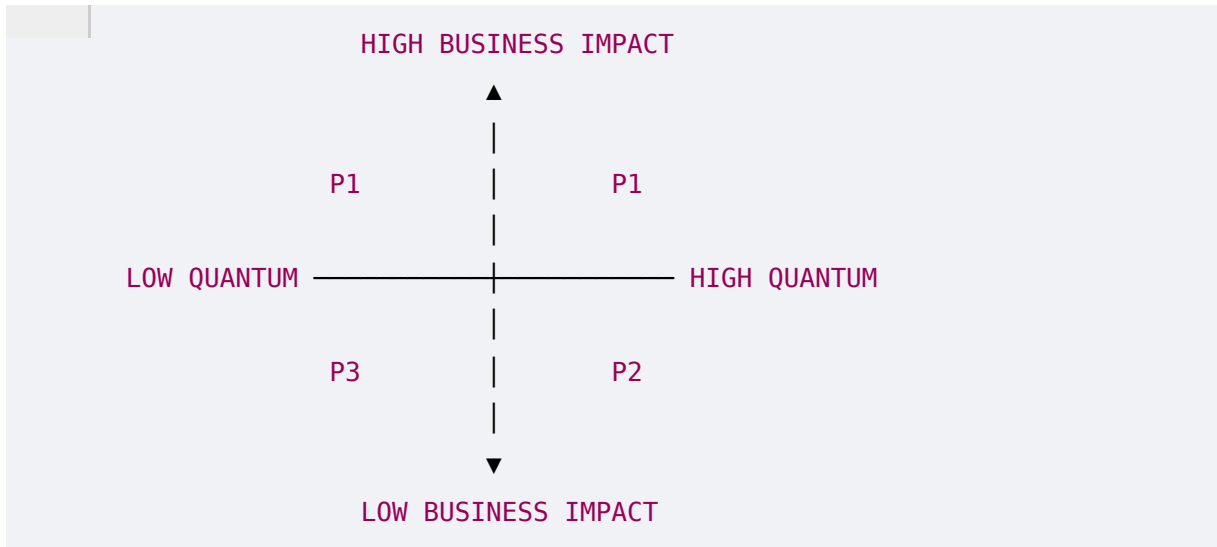
```
Business Impact:  
98
```

```
Migration Complexity:  
72
```

This preserves explainability.

39. Multi-Axis Risk

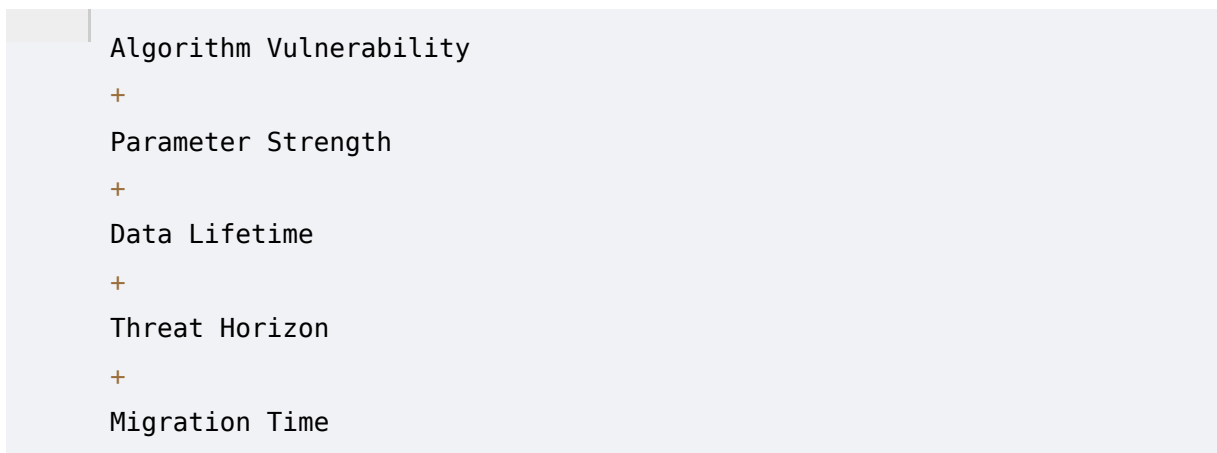
A better representation:



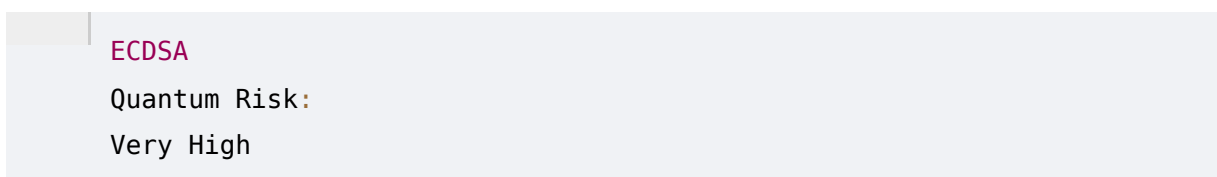
The exact prioritization policy should be configurable.

40. Quantum Risk Score

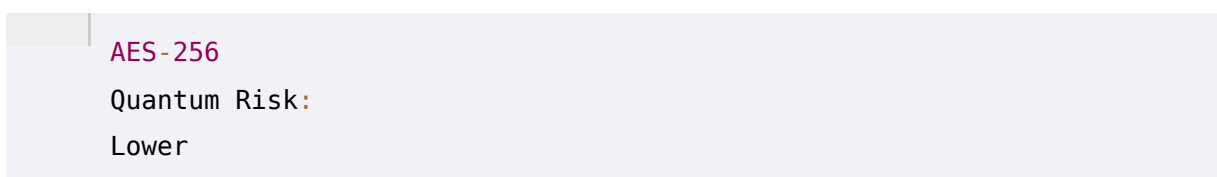
Quantum risk can incorporate:



Example:



while:



under the same scenario.

41. Business Impact Score

Possible factors:

Business Criticality

+

Data Sensitivity

+

Number of Users

+

Regulatory Importance

+

Service Availability

Example:

Government Identity System

Business Impact:

Critical

42. Migration Difficulty Score

Possible factors:

Code Complexity

+

Dependencies

+

Vendor Constraints

+

Hardware

+

Protocol

+

Testing

+

Certification

Example:

Legacy HSM

Migration Difficulty:

Very High

43. Exposure Score

Potential classification:

Internet:

High

Partner:

Medium-High

Internal:

Medium

Isolated:

Low

Again, this is one dimension rather than the entire risk.

44. Confidence Score

ECDAT should separately report:

Risk:

High

Confidence:

0.92

This means:

The system is highly confident in its assessment.

Do not confuse:

Risk severity

with:

Confidence

45. Priority Score

Priority answers:

What should we do first?

Risk answers:

How dangerous is this?

These are related but not identical.

For example:

```
Asset A:  
Risk = Critical  
Migration Difficulty = Very High  
  
Asset B:  
Risk = High  
Migration Difficulty = Low
```

Asset B may be the faster immediate remediation.

46. Risk Classification

A useful classification:

```
CRITICAL  
HIGH  
MEDIUM  
LOW  
UNKNOWN
```

47. Critical

Potential conditions:

```
Quantum-vulnerable  
+  
Critical data  
+  
Long lifetime  
+  
Migration time overlaps threat horizon
```

Example:

Critical Government Data

Protected by ECDSA

Migration Time:

5 years

Mosca:

Action Required

48. High

Example:

Quantum-vulnerable

+

Important application

+

Moderate migration complexity

Migration should be planned soon.

49. Medium

Example:

Quantum-vulnerable

+

Low sensitivity

+

Short data lifetime

May not require immediate migration but should be tracked.

50. Low

Example:

Low-impact development environment

+

Short-lived data

+

Easy replacement

Still inventory it.

51. Unknown

Unknown is important.

Example:

```
Algorithm:
Unknown

Data Lifetime:
Unknown

Migration Time:
Unknown
```

ECDAT should not manufacture a score.

Instead:

```
Risk:
UNKNOWN

Action:
Collect More Evidence
```

52. Risk Aggregation

Risk exists at multiple levels.

```
Asset
↓
Application
↓
Business Process
↓
Business Unit
↓
Enterprise
```

53. Asset-Level Risk

Example:

ECDSA P-256
Quantum Risk:
High

54. Application-Level Risk

Suppose:

Payment Application

uses:

ECDSA
RSA
AES-256

Application risk can be aggregated from its cryptographic dependencies.

55. Business-Level Risk

Suppose:

Payment Processing

depends on:

Payment Gateway
Certificate
HSM
RSA
ECDSA

The business process inherits relevant risks.

56. Enterprise-Level Risk

ECDAT can calculate:

Enterprise Quantum Exposure

using:

```
Number of affected systems
+
Criticality
+
Data sensitivity
+
Migration progress
```

57. Risk Propagation

Example:

```
ECDSA
↓
Certificate
↓
Payment API
↓
Payment Process
```

If ECDSA becomes high risk:

```
Potential impact
propagates upward.
```

But the engine must not blindly multiply risk.

Each relationship should define how risk propagates.

58. Dependency Risk

Example:

```
OpenSSL
↓
2,000 Applications
```

A cryptographic issue in the library can create:

```
Systemic Risk
```

This is different from a weakness affecting one application.

59. Shared Infrastructure Risk

Example:

HSM-01

↓

500 Applications

If HSM-01 does not support required PQC operations:

Migration Bottleneck:

Critical

The problem is not merely cryptographic.

It is architectural.

60. PKI Risk

Suppose:

Root CA

↓

5 Intermediate CAs

↓

100,000 Certificates

If the PKI ecosystem uses quantum-vulnerable public-key mechanisms:

Enterprise Migration Scope:

Huge

ECDAT should surface this concentration.

61. HSM Risk

An HSM may be:

Quantum-ready

Quantum-transition-ready

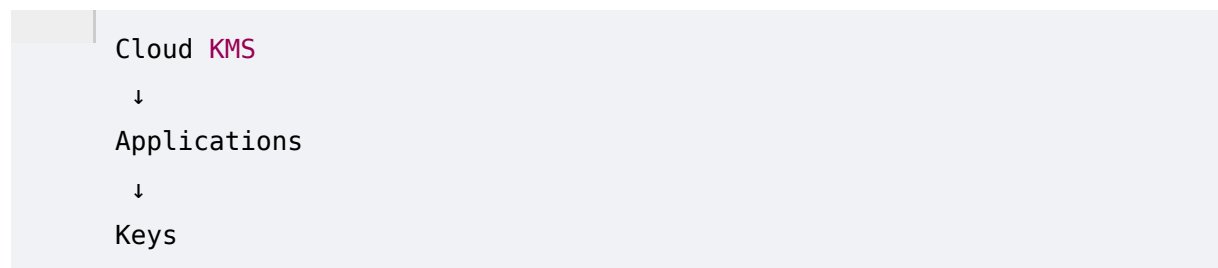
Legacy

Unknown

The organization may need hardware upgrades before applications can migrate.
This becomes a migration dependency.

62. Cloud KMS Risk

Similarly:



If PQC support is unavailable or constrained, migration may depend on provider capabilities.

ECDAT should represent this as a constraint rather than simply marking the application "high risk."

63. Data-Centric Risk

This is arguably the most important risk perspective.

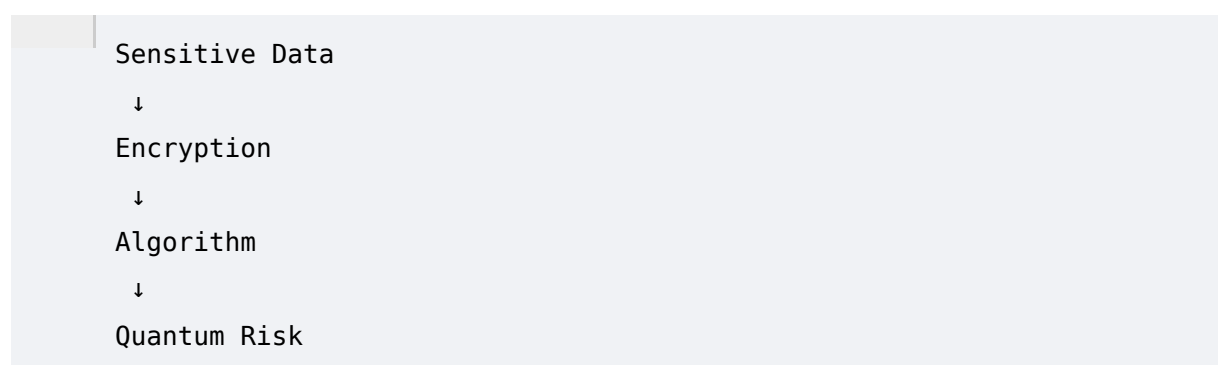
Instead of:

Which algorithms are vulnerable?

ask:

Which sensitive data is protected by cryptography that may require migration?

Graph:



64. Scenario Analysis

ECDAT should allow different scenarios.

Scenario A

Quantum threat horizon:
25 years

Scenario B

Quantum threat horizon:
15 years

Scenario C

Quantum threat horizon:
10 years

The resulting priorities may change.

65. What-If Analysis

Example:

What happens if we assume the quantum threat horizon is 10 years instead of 15?

ECDAT recalculates:

Mosca
+
Risk
+
Priority

without changing the underlying assets.

This is powerful for planning.

66. Migration Prioritization

The final purpose of risk scoring is prioritization.

ECDAT should produce:

P1
P2
P3
P4

or another configurable prioritization system.

67. Priority Matrix

Example:

Quantum Risk	Business Impact	Priority
High	Critical	P1
High	High	P1
High	Medium	P2
Medium	Critical	P2
Medium	Medium	P3
Low	Low	P4

This is only an example policy.

The organization should be able to configure it.

68. P1 / P2 / P3 / P4

P1

Immediate strategic action.

P2

Near-term migration planning.

P3

Track and prepare.

P4

Monitor.

69. Risk Register

ECDAT should maintain a structured risk register.

Example:

RISK-001
Asset: ECDSA P-256
Application: Payment Gateway
Risk: Critical
Quantum: High
Business: Critical
Mitigation:

70. Risk Explanation

Every risk should have a human-readable explanation.

Bad:

Risk Score: 94

Good:

ECDSA P-256 is used by a critical payment application. The protected data has a long retention period and the estimated migration time overlaps the selected quantum threat scenario. Migration should therefore begin during the current planning cycle.

71. Evidence

The explanation should link to evidence:

```
src/security/signing.py:42
Certificate: cert-123
Application: payment-api
Data Classification: Restricted
Retention: 12 years
```

This makes the result auditable.

72. Confidence

Example:

```
Risk:
Critical

Confidence:
91%
```

Reasons:

```
Algorithm:
Directly observed

Application criticality:
CMDB evidence

Data lifetime:
Policy evidence

Migration time:
Engineering estimate
```

73. Risk Changes

ECDAT should detect:

```
Risk Increased
Risk Reduced
New Risk
```

Risk Resolved
Risk Reopened

Example:

ECDSA
Risk:
Critical

↓ migration

Risk:
Medium

74. Historical Risk

ECDAT should preserve:

2026:
Critical

2027:
High

2028:
Medium

2029:
Resolved

This allows organizations to demonstrate progress.

75. Risk Dashboard

A high-level dashboard:

QUANTUM RISK OVERVIEW	
Critical	84
High	391
Medium	1,284

Low	5,021	
Unknown	219	
P1 Assets	112	
PQC Ready	27%	

76. Executive View

Executives should see:

- Total Quantum Exposure
- Critical Business Systems
- Sensitive Data at Risk
- Migration Progress
- Top Migration Blockers
- Estimated Investment

Not thousands of raw code findings.

77. Technical View

Security teams need:

- Algorithm
- Key
- Certificate
- Library
- Application
- Evidence
- Risk
- Dependency
- Recommendation

78. Developer View

Developers need:

- File
- Line
- Function
- Detected Algorithm

Why It Matters
Recommended Change

Example:

```
src/signing.py:42
```

Detected:

ECDSA P-256

Recommendation:

Use an approved PQC-compatible signature abstraction.

79. Risk API

Example:

```
GET /api/risks
```

Filters:

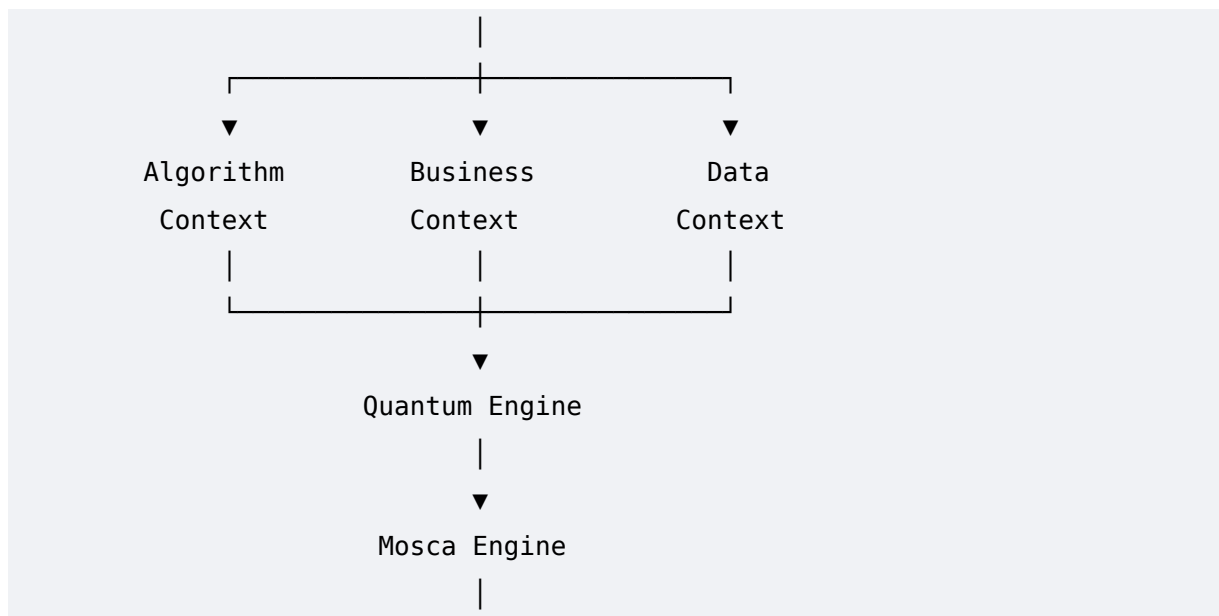
```
severity=critical  
quantum=true  
application=payment-api
```

Potential response:

```
{  
  "risk_id": "risk-001",  
  "severity": "critical",  
  "quantum": true,  
  "priority": "P1",  
  "confidence": 0.92  
}
```

80. Risk Engine Architecture

CBOM
|
▼
KNOWLEDGE GRAPH



81. Complete Risk Pipeline



82. Example 1 — ECDSA Payment System

Discovery



Context

Application:
Payment Gateway

Business Criticality:
Critical

Data Lifetime:
12 years

Migration Time:
5 years

Threat Scenario

Threat Horizon:
15 years

Mosca

$12 + 5 > 15$
 $17 > 15$

Result

Quantum Risk:
High

Priority:
P1

Recommendation:
Begin migration planning

83. Example 2 — AES-256 Database

Algorithm:
AES-256-GCM

Data:

Internal Records

Lifetime:

3 years

The quantum posture is materially different from public-key cryptography.

ECDAT might produce:

Quantum Risk:

Low / Context-Dependent

Action:

Monitor

Priority:

P3

The exact result should depend on the organization's policy.

84. Example 3 — RSA Certificate

Certificate:

RSA-2048

Application:

Public API

Validity:

3 years

Data:

Sensitive

Migration:

2 years

ECDAT:

Quantum:

High

Business:

High

Migration:

Moderate

Priority:

P1/P2

depending on organizational thresholds.

85. Example 4 — Unknown Custom Crypto

Function:

`custom_encrypt()`

Algorithm:

Unknown

Confidence:

0.58

Data:

Restricted

ECDAT should not guess.

Instead:

Risk:

UNKNOWN / POTENTIALLY HIGH

Action:

Security Review Required

Priority:

P1 investigation

This is a good example of separating:

Unknown Algorithm

from:

Known Quantum-Vulnerable Algorithm

86. Example 5 — Shared Library

OpenSSL

↓

2,400 Applications

Suppose many applications use quantum-vulnerable functionality.

ECDAT can report:

Library:

OpenSSL

Affected Applications:

2,400

Migration Centrality:

Very High

Priority:

Strategic

This may mean:

Upgrade the shared cryptographic foundation first.

87. Requirements Derived From Phase 12

ECDAT's risk engine should support:

Quantum

- Shor classification
- Grover classification
- Public-key vulnerability
- Symmetric security margin
- Hash considerations

Mosca

- Data lifetime
- Migration time
- Threat horizon
- Scenario analysis
- Uncertainty

Context

- Business criticality
- Data sensitivity
- Exposure
- Dependency centrality
- Migration complexity

Risk

- Asset risk
- Application risk
- Business risk
- Enterprise risk
- Priority

Explainability

- Evidence
- Confidence
- Reasoning
- Historical changes

Visualization

- Risk matrix
- Risk heatmap
- Priority dashboard
- Business impact
- Migration blockers

88. Phase 12 Summary

The fundamental transformation is:

BEFORE

ECDSA detected.

After Phase 12:

AFTER

ECDSA P-256 detected.

Used by:

Payment Gateway

Protects:

Sensitive financial data

Data lifetime:

12 years

Estimated migration:

5 years

Quantum scenario:

That is the difference between:

cryptographic inventory

and:

cryptographic risk intelligence.

89. The Most Important Insight

ECDAT should never say:

"RSA is bad."

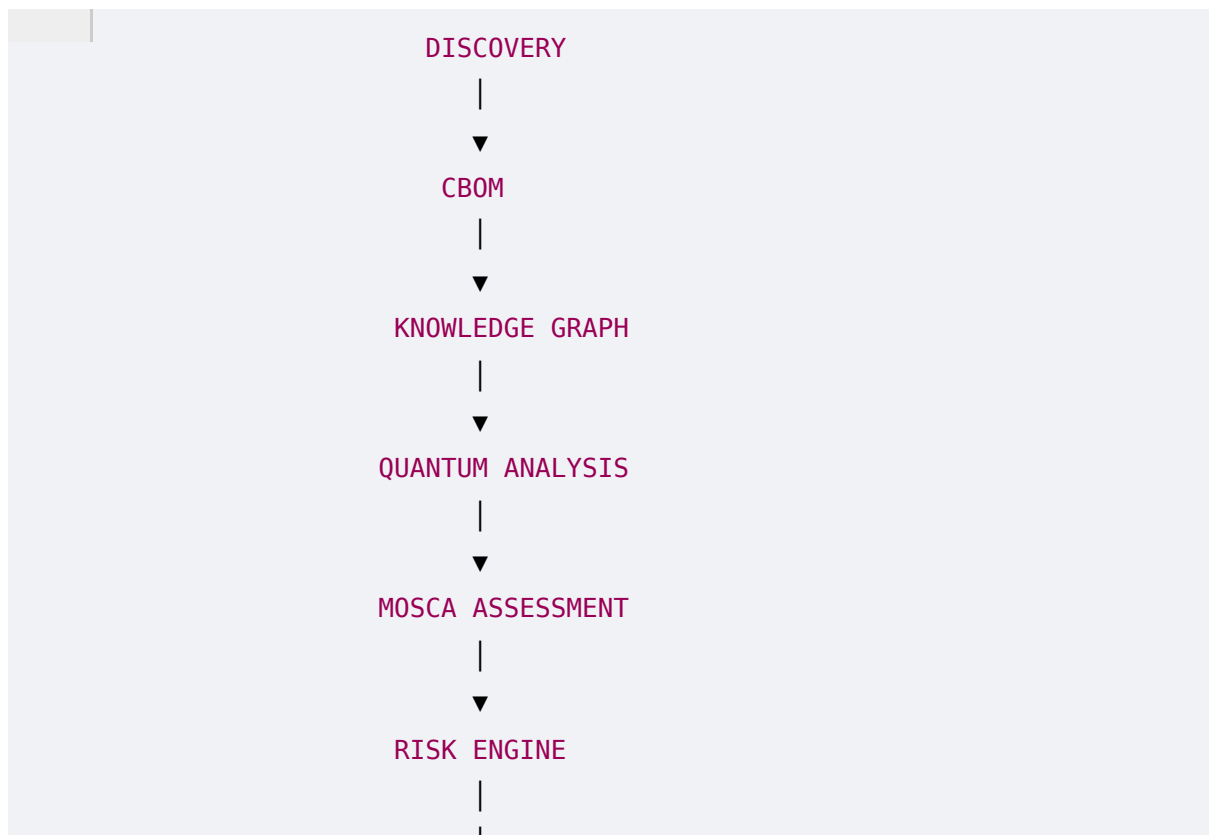
It should say:

"This specific RSA deployment creates this specific risk because of this specific algorithm, parameter, data lifetime, business context, migration timeline, and dependency structure."

That is the level of reasoning expected from an enterprise security platform.

90. ECDAT Risk Intelligence Chain

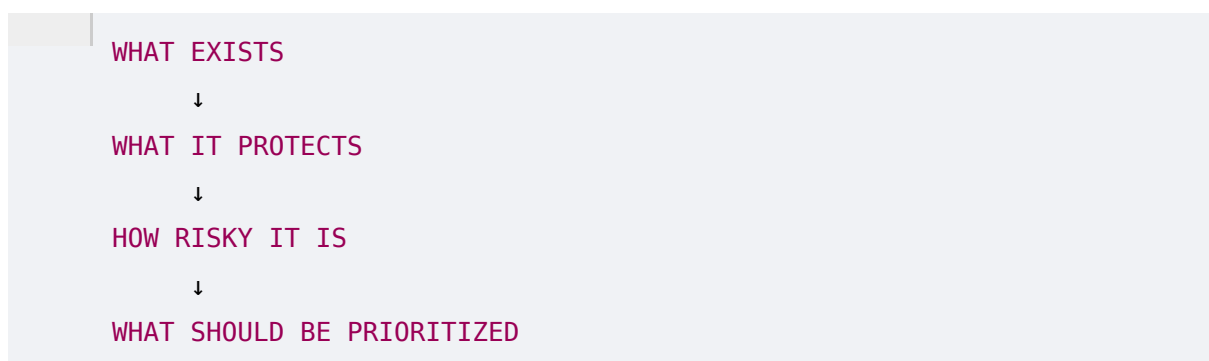
The platform now has:



This is now a genuine decision-support architecture.

PHASE 13 PREVIEW — PQC RECOMMENDATION ENGINE & CRYPTO-AGILITY

Now we know:



The next question is:

What should replace it?

The original SIH problem statement explicitly requires:

Recommend suitable alternatives (PQC/Hybrid algorithms) for applications based on risk profile, latency, cost, etc.

Phase 13 will build that recommendation engine.

We will cover:

- NIST PQC standards
- ML-KEM
- ML-DSA
- SLH-DSA
- Hybrid cryptography
- Classical + PQC combinations
- KEM architecture
- Digital signatures
- Key exchange
- Encryption
- Authentication
- TLS migration
- PKI migration
- Certificate migration
- HSM compatibility
- Library compatibility
- Hardware compatibility
- Performance
- Latency
- Memory
- Bandwidth
- Key sizes
- Signature sizes
- Interoperability
- Vendor support
- Compliance

- Application constraints
- Migration complexity
- Cost
- Recommendation scoring
- Algorithm selection matrix
- Crypto-agility
- Abstraction layers
- Provider architectures
- Policy engine
- Automated recommendations
- Human approval
- What-if comparisons
- Hybrid transition strategies
- Rollback strategies
- Migration sequencing

The output will evolve from:

```
ECDSA detected
```

to:

```
ECDSA P-256
  ↓
Risk: Critical
  ↓
Candidates:
ML-DSA
Hybrid ECDSA + ML-DSA
SLH-DSA
  ↓
Evaluate:
Security
Performance
Compatibility
Cost
Hardware
Protocol
  ↓
Recommended:
```


That is where ECDAT begins to become a **PQC migration advisor**, not just a risk scanner.

Phase 12 complete.